

Incremental Learning Algorithms in Autonomous Driving and Robot Lifelong Skill Acquisition

1. Introduction

Autonomous systems, including self-driving vehicles and intelligent robotic agents, operate in **dynamic, non-stationary environments** where the distribution of data can change continuously over time. Traditional machine learning models are trained in a **static setting**, assuming access to all training data at once. This assumption is unrealistic for deployed systems, as new objects, tasks, and scenarios may emerge post-deployment. Re-training models from scratch for each update is computationally expensive, data-intensive, and often infeasible in safety-critical applications.

Incremental Learning (IL), also known as Continual Learning (CL), addresses this limitation by enabling models to **learn sequentially from new data while retaining knowledge from previous tasks**. A successful incremental learning system must balance:

- **Plasticity:** Ability to learn new tasks effectively
- **Stability:** Ability to retain knowledge from prior tasks
- **Efficiency:** Minimal memory, computation, and deployment overhead
- **Safety and Robustness:** Especially in autonomous driving, where errors can be catastrophic

In robotics, IL also enables **lifelong skill acquisition**, allowing robots to incrementally learn manipulation, navigation, and sensorimotor tasks without human supervision or reprogramming. This chapter surveys the **major incremental learning algorithms** applied in autonomous driving and robot lifelong skill acquisition, detailing:

1. Algorithmic mechanisms
2. Dataset preparation and design for incremental training
3. Practical utility in autonomous driving and robotic systems
4. Advantages, limitations, and research perspectives

2. Replay-Based Incremental Learning Methods

Replay methods mitigate forgetting by **storing and reusing a subset of past experiences**. They are widely used in **perception-heavy systems**, where replayed exemplars or synthetic data guide the model to retain prior knowledge.

2.1 iCaRL (Incremental Classifier and Representation Learning)

Mechanism:

iCaRL handles **class-incremental learning** by maintaining a fixed-size **memory buffer of exemplars** for previously learned classes. Training on a new task includes both new data and replayed exemplars. The model uses **nearest-mean-of-exemplars (NME) classification**, computing the mean feature vector for each class from stored exemplars, which stabilizes predictions as new classes are added.

Dataset Preparation:

- Sequentially add new object categories or tasks
- Select a fixed number of **representative exemplars per class** using a herding strategy
- Maintain memory size while balancing all classes

Autonomous Driving Utility:

- Incremental traffic sign recognition (e.g., newly deployed signs)
- Updating object detection modules for new vehicle types or road objects
- Adaptation to city-specific object distributions

Robotics Utility:

- Incremental object recognition for manipulation tasks
- Visual memory for grasping or assembly objects

Advantages:

- Strong performance on class-incremental tasks
- Explicit memory retention via exemplars

Limitations:

- Requires memory to store samples
 - Exemplar selection can be computationally expensive
-

2.2 GDumb (Greedy DUMB Learner)

Mechanism:

GDumb is a surprisingly strong baseline that **discards old model weights** and maintains a **balanced memory buffer** of samples from all tasks. At each incremental step, the model is **retrained from scratch** using only this memory, avoiding catastrophic forgetting entirely by design.

Dataset Preparation:

- Maintain a fixed-size buffer
- Greedily select new samples while maintaining class balance

- No need to track parameter importance or task order

Autonomous Driving Utility:

- Offline retraining of perception modules
- Safe adaptation to updated object distributions
- Benchmarking and strong baseline for incremental perception studies

Robotics Utility:

- Consolidation of skill datasets in simulation
- Retraining policies from stored trajectories in episodic tasks

Advantages:

- Simplicity and strong empirical results
- Avoids forgetting without complex mechanisms

Limitations:

- Requires retraining from scratch
 - Limited feasibility for online deployment
-

2.3 DER / DER++ (Dark Experience Replay)

Mechanism:

DER enhances classical replay by storing both **input samples and their logits**. During incremental learning, the loss combines classification loss for new data and **distillation loss** to match stored logits, preserving old knowledge. DER++ further incorporates **ground-truth labels** for replayed samples, improving stability.

Dataset Preparation:

- Store input–logit pairs for previous tasks
- Fixed memory buffer size
- Can complement new-task datasets with replayed examples

Autonomous Driving Utility:

- Incremental semantic segmentation across cities, lighting, or weather
- Continuous improvement of perception systems while maintaining old knowledge

Robotics Utility:

- Continual learning for sensorimotor mappings
- Retaining learned policies during incremental skill acquisition

Advantages:

- Combines replay with knowledge distillation
- Memory-efficient compared to full dataset storage

Limitations:

- Requires storage of logits
 - Complexity higher than naive replay
-

3. Regularization-Based Incremental Learning Methods

Regularization-based methods avoid storing data by **penalizing changes to important parameters**, constraining the network to retain prior knowledge.

3.1 EWC (Elastic Weight Consolidation)

Mechanism:

EWC computes the **Fisher Information Matrix** to estimate the importance of each network parameter for previously learned tasks. The loss function for a new task includes a **quadratic penalty** for changing parameters with high importance.

Dataset Preparation:

- Sequential task datasets
- Parameter importance computed at task boundaries
- No replay buffer needed

Autonomous Driving Utility:

- Experimental incremental adaptation for perception
- Limited by highly overlapping and continuous task domains

Robotics Utility:

- Sequential reinforcement learning tasks
- Preserving previously learned locomotion or manipulation policies

Advantages:

- Memory-efficient
- No need to store old samples

Limitations:

- Poor scalability to large numbers of tasks

- Sensitive to hyperparameter tuning
-

3.2 SI (Synaptic Intelligence)

Mechanism:

SI tracks **parameter contributions online** by accumulating how much each parameter reduces the loss during training. Important parameters are penalized during updates for new tasks.

Dataset Preparation:

- Continuous stream of task data
- Online importance computation
- No task boundaries or labels required

Autonomous Driving Utility:

- Limited, mainly for controlled experiments
- Could be applied for incremental sensor adaptation

Robotics Utility:

- Online skill acquisition in continuous learning settings
- Memory-constrained embedded robots

Advantages:

- Task-boundary independent
- Lightweight memory footprint

Limitations:

- Less effective for complex perception tasks
-

3.3 LwF (Learning without Forgetting)

Mechanism:

LwF uses **knowledge distillation** from a previous model to preserve old knowledge. The current model is trained on new-task data while matching the teacher's predictions for old tasks.

Dataset Preparation:

- Only new-task data required
- Teacher generates soft targets for distillation

Autonomous Driving Utility:

- Updating perception modules without access to old data

- Regression prevention in model updates

Robotics Utility:

- Policy updates without storing old trajectories
- Knowledge transfer between robot agents

Advantages:

- No data storage needed
- Flexible for various modalities

Limitations:

- Performance depends on teacher predictions
 - Less robust than replay-based methods
-

3.4 MAS (Memory Aware Synapses)

Mechanism:

MAS estimates parameter importance by measuring **sensitivity of output function** to each parameter. Highly influential parameters are constrained during new task learning.

Dataset Preparation:

- Can use unlabeled data
- Online computation of parameter importance

Autonomous Driving Utility:

- Experimental applications for self-supervised perception adaptation

Robotics Utility:

- Continuous learning for manipulation and navigation
- Low-labeled data scenarios

Advantages:

- Can operate with unlabeled streams
- Suitable for online learning

Limitations:

- Requires careful sensitivity computation
 - Less common in large-scale AD systems
-

4. Parameter Isolation and Architecture Growth Methods

These methods **structurally prevent forgetting** by isolating task-specific parameters or dynamically expanding the network.

4.1 PackNet

Mechanism:

PackNet prunes and freezes network weights after learning each task, reusing freed parameters for new tasks. This creates **isolated subnetworks** per task.

Dataset Preparation:

- Task-specific datasets
- Post-task pruning and masking

Autonomous Driving Utility:

- Rare due to latency and certification constraints

Robotics Utility:

- Modular skill libraries
- Safe incremental skill learning

Advantages:

- Zero forgetting by design
- Efficient reuse of parameters

Limitations:

- Pruning and masking overhead
 - Memory allocation challenges
-

4.2 Progressive Neural Networks

Mechanism:

Each new task is assigned a **new column** in the network, previous columns are frozen, and **lateral connections** allow feature reuse.

Dataset Preparation:

- One dataset per task
- Column growth triggered by new tasks

Autonomous Driving Utility:

- Impractical due to unbounded growth

Robotics Utility:

- Sequential skill learning
- Zero-forgetting reinforcement learning

Advantages:

- Guaranteed retention
- Task-specific specialization

Limitations:

- Model growth becomes large
 - Computational overhead
-

4.3 DEN (Dynamically Expandable Networks)

Mechanism:

DEN selectively expands the network only when existing parameters fail to represent new tasks adequately. **Selective retraining** balances reuse and expansion.

Dataset Preparation:

- Sequential task datasets
- Expansion triggered by performance threshold

Autonomous Driving Utility:

- Experimental studies for incremental perception

Robotics Utility:

- Long-horizon skill acquisition
- Adaptive model complexity

Advantages:

- Flexible capacity allocation
- Controlled forgetting

Limitations:

- Increased architecture complexity
 - Sensitive to expansion criteria
-

5. Prompt and Adapter-Based Methods

5.1 Prompt Tuning

Mechanism:

Small **task-specific prompt vectors** are learned and prepended to inputs while the backbone is frozen. The prompts guide the model for new tasks.

Dataset Preparation:

- Task-specific datasets
- Pretrained backbone

Autonomous Driving Utility:

- Multi-modal scene understanding
- Vision-language perception updates

Robotics Utility:

- Instruction-based skill learning
 - Multi-task robotic instruction generalization
-

5.2 Adapter Modules

Mechanism:

Small trainable layers inserted between frozen backbone layers. Each task updates only its adapters.

Dataset Preparation:

- Unified input format across tasks
- Task-specific adapter parameters

Autonomous Driving Utility:

- Incremental perception module updates
- Safe backbone freezing

Robotics Utility:

- Efficient multi-task skill acquisition
 - Memory-efficient lifelong learning
-

6. Generative Replay Methods

6.1 Generative Replay

Mechanism:

Generative models (GANs, VAEs, diffusion models) synthesize previous task data for replay. The synthesized data is used alongside new-task data to prevent forgetting.

Dataset Preparation:

- Train generative model per task
- Use generated samples in incremental learning

Autonomous Driving Utility:

- Synthetic rare-event generation (e.g., accident scenarios)
- Safe data augmentation for perception

Robotics Utility:

- Simulation-based skill rehearsal
- Memory-efficient replay for long task sequences

7. Comparative Analysis (Scientific Metrics)

Algorithm	Memory Cost	Computation Cost	Scalability	Utility in Autonomous Driving (AD)	Utility in Robot Lifelong Skill Acquisition
iCaRL	$O(k \cdot d)$ exemplar memory (typically 1–5% of full dataset; k exemplars per class, d feature dim)	$O(N \cdot d)$ per task (standard training + nearest-mean inference)	Moderate – degrades as class count grows	Incremental traffic sign, object, and lane-marking classification	Long-term object recognition for manipulation and navigation
GDumb	$O(M)$ fixed buffer (M samples, usually ≤ 5 –10% of dataset)	High: $O(T \cdot M)$ full retraining from scratch per task	Moderate – limited by retraining cost	Offline perception model refresh	Skill consolidation from curated experience
DER / DER+ +	$O(M \cdot (x+z))$ buffer storing inputs + logits/features	Medium – joint loss adds ~ 10 –20% FLOPs	Good – stable with many tasks	Incremental semantic segmentation, detection	Policy/value retention in continual RL

Algorithm	Memory Cost	Computation Cost	Scalability	Utility in Autonomous Driving (AD)	Utility in Robot Lifelong Skill Acquisition
EWC	O(P) Fisher diagonal ($\approx 1 \times$ model parameters)	Low – quadratic penalty adds negligible FLOPs	Limited – assumes task separation	Experimental use in AD control policies	Sequential task learning in RL
SI (Synaptic Intelligence)	O(P) parameter importance traces	Low – lightweight online updates	Moderate – better than EWC in online settings	Experimental perception adaptation	Online motor skill acquisition
LwF	O(1) (no data storage, logits only at training time)	Medium – extra forward pass for distillation	Moderate – struggles with large domain shifts	Incremental perception (objects, drivable area)	Policy updates without replay
MAS	O(P) importance weights via gradients	Medium – gradient-based importance estimation	Moderate	Experimental AD perception/control	Manipulation & navigation skills
PackNet	O(P) masks per task (binary masks ~ 1 bit/param/task)	Medium – pruning + retraining	Moderate – capacity saturates	Limited applicability	Long-term skill libraries
Progressive Neural Networks	O(T·P) linear growth with tasks	High – parallel column inference	Poor – not scalable	Limited real-time feasibility	Skill accumulation without forgetting
DEN	O(P + ΔP) dynamic expansion ($\Delta P \approx 5\text{--}20\%$ per task)	High – expansion + regularization	Moderate	Experimental AD learning	Adaptive skill learning
Prompt Tuning	O(T·p) prompts ($p \ll P$, usually $< 0.1\%$ of model)	Low – backbone frozen	High – scales to many tasks	Multi-modal perception (VLM-based AD)	Instruction-based lifelong learning
Adapter Modules	O(T·a) adapters ($a \approx 1\text{--}5\%$ of backbone)	Low – small extra layers	High	Incremental perception stacks	Multi-task robotic skills
Generative Replay	O(P_gen + M) generator + optional buffer	High – generation + training overhead	Moderate – generator limits scaling	Synthetic rare-event replay (accidents)	Simulation-based rehearsal of skills

Where:

Memory Cost Parameters

- **k** – Number of exemplars per class stored in memory (used in iCaRL).
- **d** – Feature dimensionality of the network (size of output from embedding layer).
- **M** – Total number of samples stored in a buffer (used in GDumb, DER, generative replay).
- **x** – Size of input sample (e.g., number of pixels $\times$ channels).
- **z** – Size of stored logits or feature representation per sample.
- **P** – Total number of network parameters (weights + biases).
- ΔP – Additional parameters added per task in expandable networks (DEN).
- **T** – Number of tasks learned sequentially.
- **p** – Size of task-specific prompt vectors (Prompt Tuning).
- **a** – Size of task-specific adapters (Adapter Modules).
- **P_gen** – Number of parameters in the generative model (GAN, VAE, diffusion).

Computation Cost Parameters

- **N** – Number of new samples per task.
- **d** – Feature dimensionality (used in nearest-mean computation, iCaRL).
- **FLOPs** – Floating point operations; approximate measure of computation per forward/backward pass.

Scalability Metrics

- **Moderate/Good/Poor/High/Limited** – Qualitative measure of how well the algorithm scales with increasing:
 - **Number of tasks (T)**
 - **Model size (P)**
 - **Dataset size (N, M)**
- This reflects both **memory growth** and **computational overhead**.

Domain-Specific Terms

- **AD** – Autonomous Driving
- **RL** – Reinforcement Learning
- **VLM** – Vision-Language Model (used in prompt tuning)

How to Interpret Formulas in Memory/Computation

1. **Memory Cost $O(k \cdot d)$** → iCaRL: storing k exemplars per class, each with feature vector of size d
2. **Memory Cost $O(M \cdot (x+z))$** → DER: storing M samples, each with input (x) and logits/features (z)
3. **Memory Cost $O(P)$** → EWC, SI, MAS: storing importance weights or Fisher diagonals for each network parameter
4. **Memory Cost $O(T \cdot P)$** → Progressive NN: network grows linearly with number of tasks T
5. **Computation Cost $O(N \cdot d)$** → iCaRL: cost proportional to number of new samples N and feature dimension d
6. **Computation Cost $O(T \cdot M)$** → GDumb: full retraining on M buffered samples for T tasks

8. Existing datasets

Algorithm	Example Datasets	Domain & Relevance
iCaRL	CIFAR-100, ImageNet-Subset, CORe50	Used for class-incremental learning experiments; CORe50 is particularly for robotic object recognition in continual learning . Autonomous driving adaptation can use KITTI/BDD100K subsets for incremental object classes.
GDumb	CIFAR-100, TinyImageNet, CORe50	Evaluates offline replay strategies; also used in robotic object recognition and incremental perception tasks.
DER / DER++	CIFAR-100, ImageNet, CORe50, Cityscapes	Strong baseline for incremental segmentation (Cityscapes for AD perception) and classification (robotic objects in CORe50).
EWC	MNIST, PermutedMNIST, CIFAR-100, CARLA synthetic datasets	Classical RL/vision tasks for testing regularization-based forgetting prevention; CARLA provides AD-like perception scenarios.
SI (Synaptic Intelligence)	PermutedMNIST, CORe50, OpenAI Gym tasks	Used for online RL skill acquisition and continuous robotic learning.
LwF (Learning without Forgetting)	ImageNet, CORe50, KITTI/BDD100K	Incremental object classification; KITTI/BDD100K for perception module updates in autonomous driving .
MAS	CIFAR-100, CORe50, OpenAI Gym	Online skill acquisition, manipulation, and navigation in robotic settings.
PackNet	ImageNet, CIFAR-100, CORe50	Multi-task classification; can be applied to robotic skill libraries .
Progressive Neural Networks	Atari-2600 RL Suite, OpenAI Gym, CORe50	Sequential reinforcement learning; skill accumulation in robots and games.
DEN (Dynamically)	CIFAR-100, ImageNet subsets, CORe50	Adaptive task expansion for both classification and robotic skill acquisition.

Algorithm	Example Datasets	Domain & Relevance
Expandable Networks)		
Prompt Tuning	KITTI, BDD100K, nuScenes, Cityscapes, Robot Learning Datasets (RoboNet, RLBench)	Vision-language or multi-modal incremental learning for perception in AD and task instructions for robotics.
Adapter Modules	KITTI, BDD100K, Cityscapes, RoboNet	Incremental perception adaptation in autonomous driving and multi-task robotic skill learning.
Generative Replay	MNIST, CIFAR-100, Cityscapes, CARLA synthetic driving datasets, RLBench	Synthetic replay for rare events; used in simulation-to-real transfer in robotics and AD (e.g., accidents or unusual objects).

Notes on Datasets

- **CIFAR-100, ImageNet, TinyImageNet:** Standard benchmarks for incremental classification experiments; widely used to compare IL methods.
- **CORe50:** Designed for **robotic object recognition** with incremental object appearances and real-world variability.
- **PermutedMNIST:** Classic incremental learning benchmark for testing regularization-based methods.
- **Cityscapes, KITTI, BDD100K, nuScenes:** Autonomous driving datasets for object detection, semantic segmentation, and multi-task perception; can be split incrementally for IL experiments.
- **CARLA synthetic datasets:** Simulator-generated driving data for incremental perception and rare-event generation.
- **OpenAI Gym / Atari RL Suite / RLBench:** Used for incremental **robotic skill learning**, policy retention, and multi-task reinforcement learning.
- **RoboNet:** Multi-robot dataset for continual manipulation skill learning.
- **RLBench:** Benchmark for lifelong learning in robotic manipulation tasks.

9. Future Research Directions

- Integration of **foundation models** for lightweight incremental adaptation
- **Generative replay** combined with simulation-to-real skill transfer
- Safety and certification considerations in **autonomous driving**
- Evaluation benchmarks for **long-term robotic skill acquisition**
- Hybrid approaches combining replay, distillation, and adapters

Notes:

- see what others did
- real dataset not standard
- try to find existing projects that used ROS2 and incremental learning
- try to find innovative ideas about incremental learning algorithms